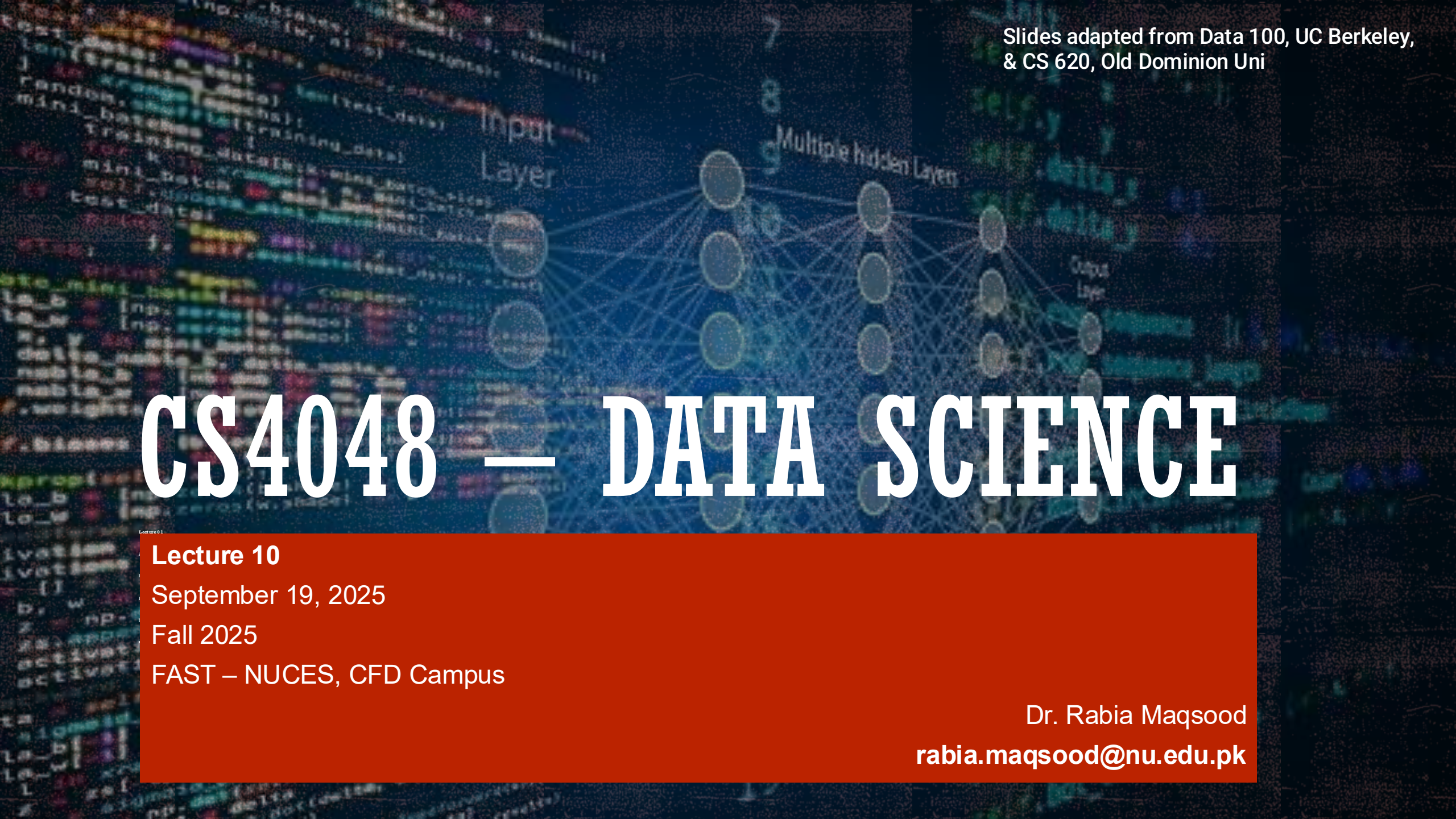


CS4048 — DATA SCIENCE



Lecture 10

September 19, 2025

Fall 2025

FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

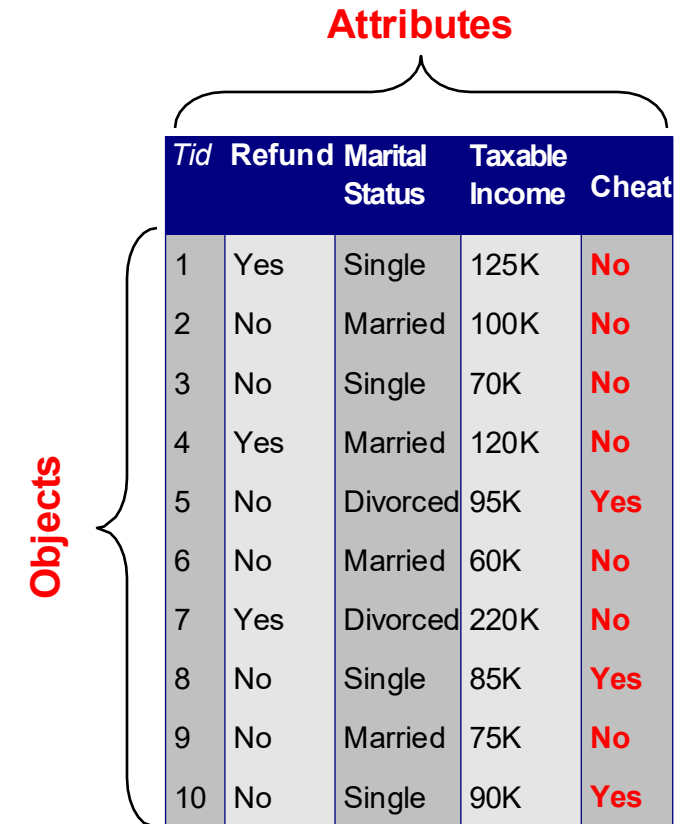
rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Data types
- Reading data in Python
- Data Wrangling/Cleaning
 - Basic cleaning
 - Missing values
 - Outliers
 - Data normalization

WHAT IS DATA?

- Collection of ***data objects*** and their ***attributes***
- An **attribute** is a property or characteristic of an object (database columns)
 - Examples: eye color of a person, temperature, etc.
 - Attribute is also known as variable, field, characteristic, dimension, or feature
- A collection of attributes describe an **object** (database rows)
 - Object is also known as record, point, case, sample, entity, or instance



Attributes

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Objects

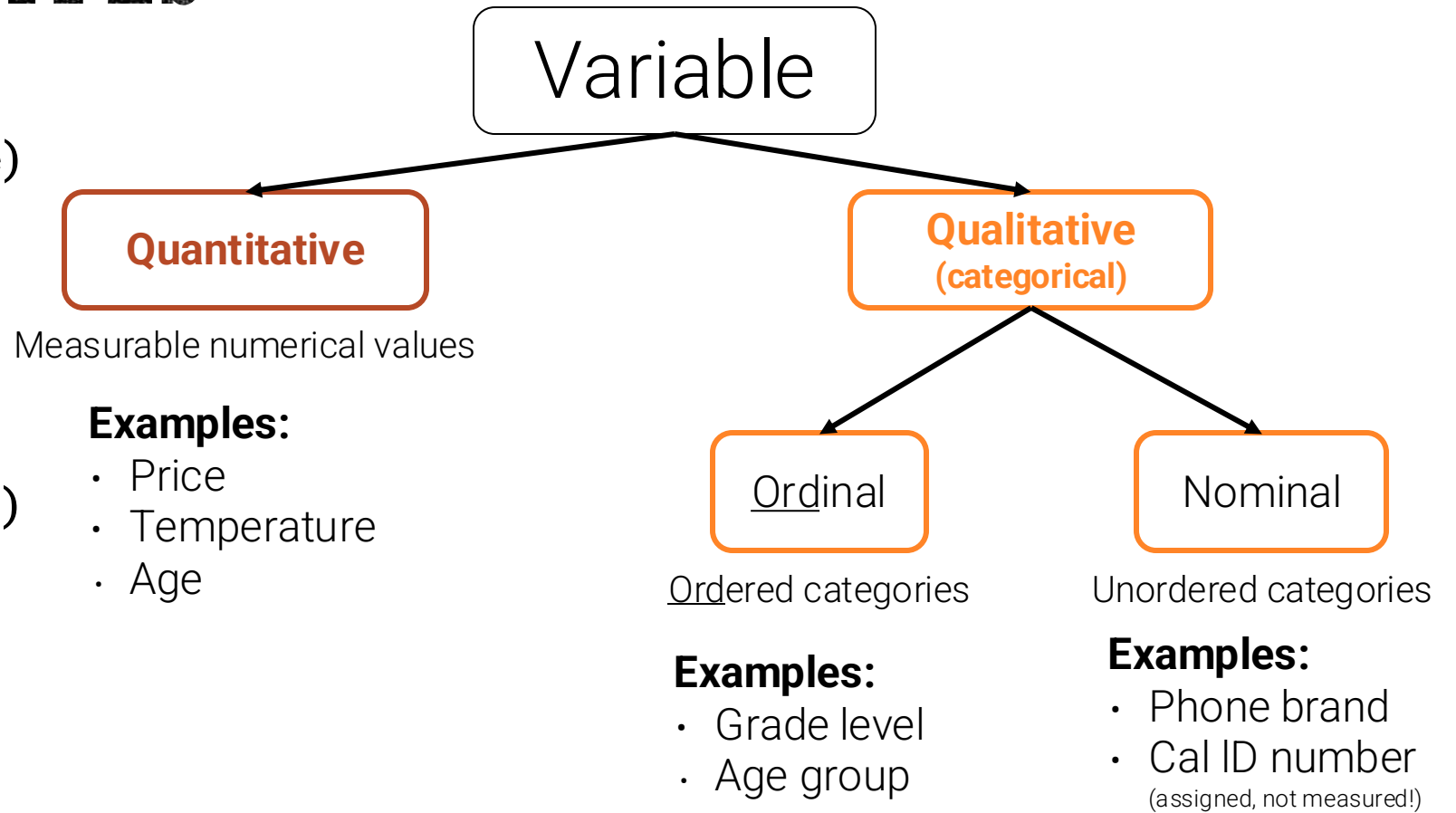
ATTRIBUTE TYPES

1. Categorical (qualitative)

- Nominal
- Binary
- Ordinal

2. Numerical (quantitative)

- Interval-scaled
- Ratio-scaled



A safe default: Store qualitative data as strings!

ATTRIBUTE TYPES

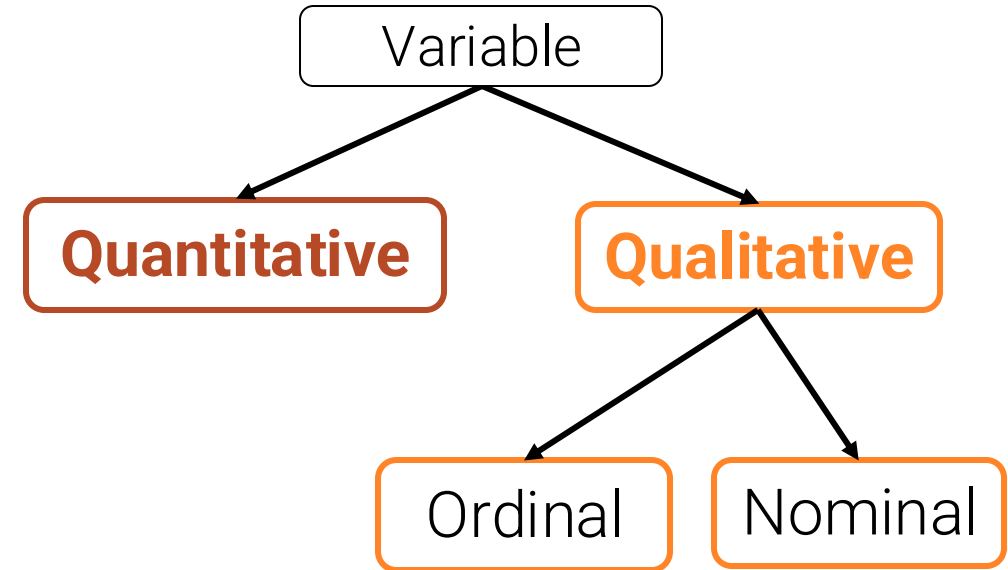
- There are two basic types of data: *numerical* and *categorical* data.
- **Numerical data:** data to which a number is assigned as a quantitative value.
 - age, weight, shoe size....
- **Categorical data:** data defined by the classes or categories into which an individual member falls.
 - eye color, gender, blood type, ethnicity

ATTRIBUTE TYPES



What is the feature type of each variable?

Q	Variable	Feature Type
1	CO ₂ level (ppm)	Quantitative
2	Income bracket (low, med, high)	Qualitative Ordinal
3	Race/Ethnicity	Qualitative Nominal
4	Political party	Qualitative Ordinal / Nominal
5	Year	Quantitative / Qualitative Ordinal
6	GPA	Quantitative / Qualitative Ordinal
7	Date and time	?



The distinction between categories is sometimes murky. Context matters!

CONTINUOUS VS. DISCRETE

- A **continuous variable** is one in which it can theoretically assume any value between the lowest and highest point on the scale on which it is being measured
 - (e.g. weight, speed, price, time, height)
- Non-continuous variables, also known as **discrete variables**, that can only take on a finite number of values
 - Discrete data can be numeric -- like numbers of apples
 - But it can also be categorical -- like red or blue, or male or female, or good or bad.

QUALITATIVE VS. QUANTITATIVE

- A **qualitative data** is one in which the “true” or naturally occurring levels or categories taken by that variable are not described as numbers but rather by verbal groupings
 - Open ended answers
- **Quantitative data** on the other hand are those in which the natural levels take on certain quantities (e.g. price, travel time)
- That is, quantitative variables are measurable in some numerical unit (e.g. pesos, minutes, inches, etc.)
 - Likert scales, semantic scales, yes/no, check box

EXPLORING YOUR DATA

- Working with data is both an art and a science. We've mostly been talking about the science part, getting your feet wet with Python tools for Data Science. Lets look at some of the art now.
- After you've identified the questions you're trying to answer and have gotten your hands on some data, you might be tempted to dive in and immediately start building models and getting answers. But you should resist this urge. Your first step should be to *explore* your data.

EXPLORING YOUR DATA

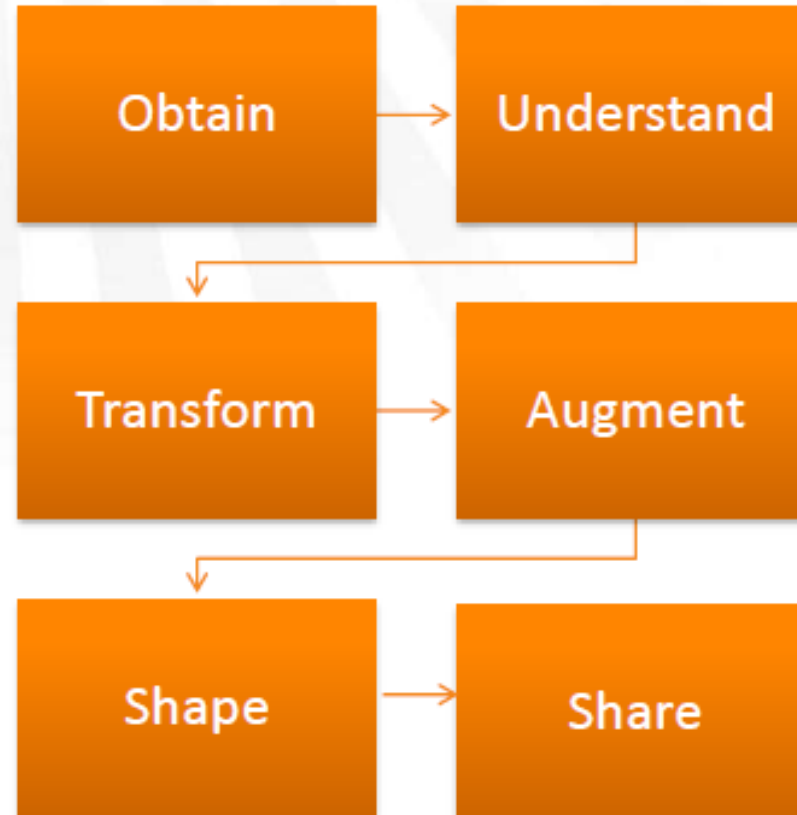


DATA WRANGLING

- The process of transforming “raw” data into data that can be analyzed to generate valid actionable insights
- Data Wrangling : aka
 - Data preprocessing
 - Data preparation
 - Data Cleansing
 - Data Scrubbing
 - Data Munging
 - Data Transformation
 - Data Fold, Spindle, Mutilate.....

DATA WRANGLING STEPS

- Iterative process of
 - Obtain
 - Understand
 - Explore
 - Transform
 - Augment
 - Visualize



DATA WRANGLING STEPS

“Looks like my V8 Chevy is running low on fuel. Didn’t I fill up just the day before?”

UNSTRUCTURED

STRUCTURED



Owner	Vehicle	Type	Fuel Level	Engine	Last Fill
AK	Chevy	Gas	5%	V8	05/04/16



15

READING DATA

READING DATA USING PANDAS

```
In [ ]: #Read csv file  
df = pd.read_csv("http://rcc.bu.edu/examples/python/data_analysis/Salaries.csv")
```

Note: The above command has many optional arguments to fine-tune the data import process.

There is a number of pandas commands to read other data formats:

```
pd.read_excel('myfile.xlsx', sheet_name='Sheet1', index_col=None, na_values=['NA'])  
pd.read_stata('myfile.dta')  
pd.read_sas('myfile.sas7bdat')  
pd.read_hdf('myfile.h5', 'df')
```

DATA FRAME DATA TYPES

Pandas Type	Native Python Type	Description
object	string	The most general dtype. Will be assigned to your column if column has mixed types (numbers and strings).
int64	int	Numeric characters. 64 refers to the memory allocated to hold this character.
float64	float	Numeric characters with decimals. If a column contains numbers and NaNs(see below), pandas will default to float64, in case your missing value has a decimal.
datetime64, timedelta[ns]	N/A (but see the datetime module in Python's standard library)	Values meant to hold time data. Look into these for time series experiments.

DATA FRAME DATA TYPES

```
In [4]: #Check a particular column type  
df['salary'].dtype
```

```
Out[4]: dtype('int64')
```

```
In [5]: #Check types for all the columns  
df.dtypes
```

```
Out[4]: rank          object  
discipline          object  
phd                 int64  
service             int64  
sex                 object  
salary              int64  
dtype: object
```

DATA FRAMES ATTRIBUTES

Python objects have *attributes* and *methods*.

df.attribute	description
dtypes	list the types of the columns
columns	list the column names
axes	list the row labels and column names
ndim	number of dimensions
size	number of elements
shape	return a tuple representing the dimensionality
values	numpy representation of the data

DATA FRAMES METHODS

Unlike attributes, python methods have *parenthesis*.

All attributes and methods can be listed with a *dir()* function: `dir(df)`

<code>df.method()</code>	description
<code>head([n]), tail([n])</code>	first/last n rows
<code>describe()</code>	generate descriptive statistics (for numeric columns only)
<code>max(), min()</code>	return max/min values for all numeric columns
<code>mean(), median()</code>	return mean/median values for all numeric columns
<code>std()</code>	standard deviation
<code>sample([n])</code>	returns a random sample of the data frame
<code>dropna()</code>	drop all the records with missing values

SELECTING A COLUMN IN A DATA FRAME

Method 1: Subset the data frame using column name:

```
df['sex']
```

Method 2: Use the column name as an attribute:

```
df.sex
```

Note: there is an attribute *rank* for pandas data frames, so to select a column with a name "rank" we should use method 1.

AGGREGATION FUNCTIONS IN PANDAS

Aggregation - computing a summary statistic about each group, i.e.

- compute group sums or means
- compute group sizes/counts

Common aggregation functions:

min, max

count, sum, prod

mean, median, mode, mad

std, var

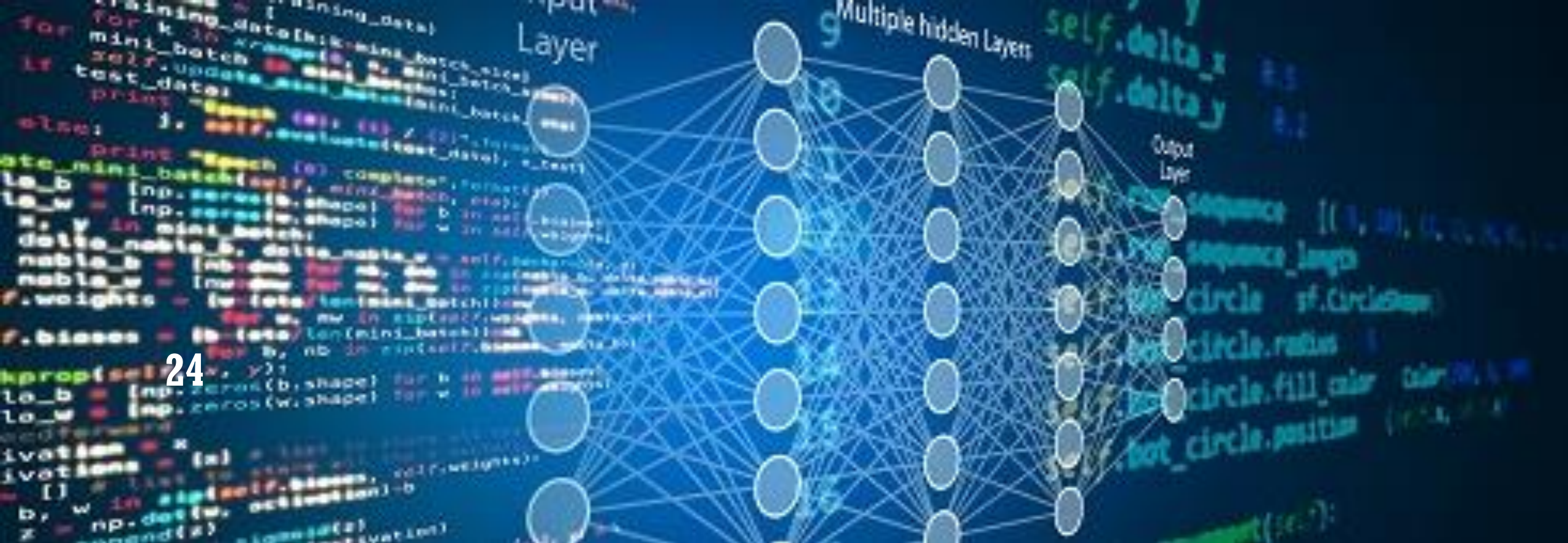
AGGREGATION FUNCTIONS IN PANDAS

agg() method are useful when multiple statistics are computed per column:

```
In [ ]: flights[['dep_delay', 'arr_delay']].agg(['min', 'mean', 'max'])
```

Out[]:

	dep_delay	arr_delay
min	-16.000000	-62.000000
mean	9.384302	2.298675
max	351.000000	389.000000



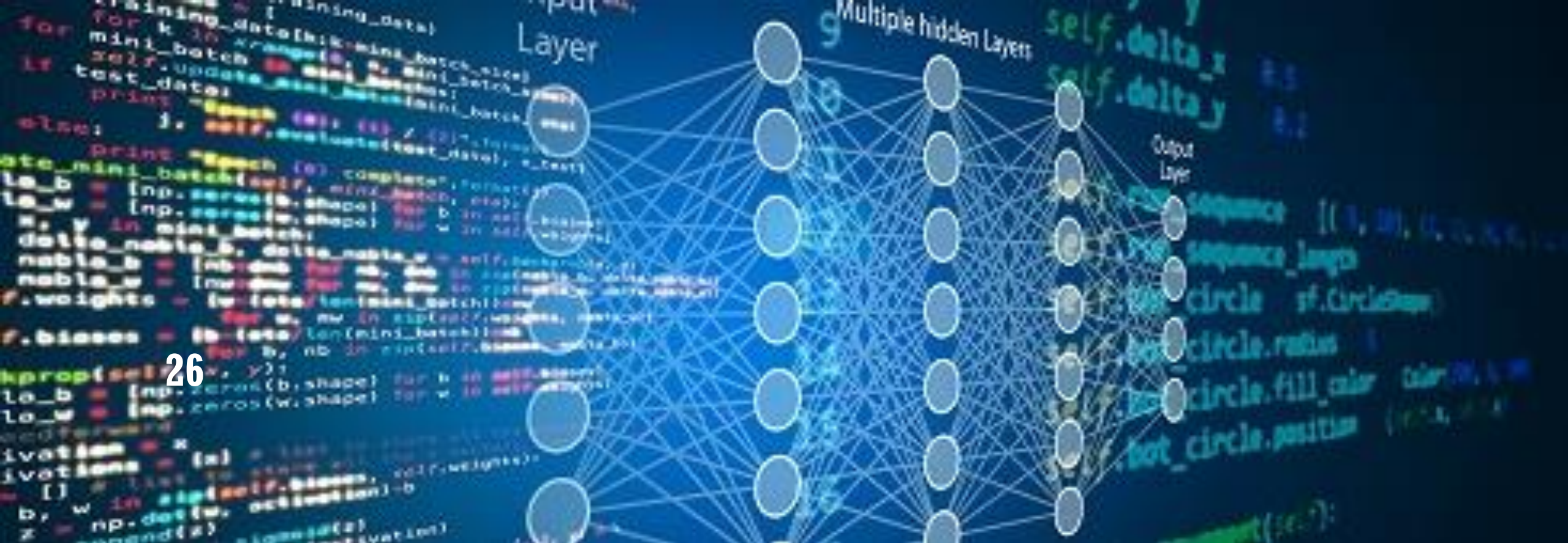
24

BASIC CLEANING

VIEW AND CLEAN DATA

- Remove duplicates
- Standardize strings/categorical data
- Check and reset data types if needed

Notebook



MISSING VALUES

MISSING VALUES

- A **missing value** is an entry that is absent or undefined in the dataset. It can arise from various sources like errors in data collection, incomplete data entry, or missing responses in surveys.
- Handling missing values correctly is crucial for accurate data analysis and modeling.
- Missing values are marked as NaN in Python

TYPES OF MISSING VALUES

1. Missing Completely at Random (MCAR): Data is missing completely at random if the **probability of a data point being missing is unrelated** to the data itself (observed) or any other variables (unobserved).

- **Example:** If an online survey randomly fails to record a few responses due to server issues, this missingness is random.
- MCAR data can be safely **ignored or imputed** without introducing bias because it doesn't depend on any patterns in the data.
- In case of imputation, values can be filled by mean/median/mode.

TYPES OF MISSING VALUES

2. Missing at Random (MAR): Data is missing at random if the **probability of missing data depends on observed data** (but not on the missing data itself).

- **Example:** In a survey, older participants may be more likely to skip questions about income, but the likelihood of missingness is related to age (observed variable) and not the income (unobserved).
- While MAR data can still be handled with techniques like **imputation**, we must **account for the relationship** between missingness and the observed variables.

TYPES OF MISSING VALUES

3. Missing Not at Random (MNAR): Data is missing not at random if the **probability of missing data depends on the missing values** themselves (i.e., the value that is missing is not random).

- **Example:** If people with very high incomes are less likely to report their income in a survey, the missingness is related to the value of the income itself.
- **Handling MNAR data is challenging.** Simple imputation methods may not work because the missingness is related to the actual values. Special methods like model-based imputation are needed.

PANDAS: HANDLING MISSING VALUES

There are a number of methods to deal with missing values in the data frame:

df.method()	description
<code>dropna()</code>	Drop missing observations
<code>dropna(how='all')</code>	Drop observations where all cells is NA
<code>dropna(axis=1, how='all')</code>	Drop column if all the values are missing
<code>dropna(thresh = 5)</code>	Drop rows that contain less than 5 non-missing values
<code>fillna(0)</code>	Replace missing values with zeros
<code>isnull()</code>	returns True if the value is missing
<code>notnull()</code>	Returns True for non-missing values

MISSING VALUES

Missing values are marked as NaN

```
In [ ]: # Read a dataset with missing values
        flights = pd.read_csv("data/flights.csv")
```

```
In [ ]: # Select the rows that have at least one missing value
        flights[flights.isnull().any(axis=1)].head()
```

```
Out [ ]:
```

	year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	hour	minute
330	2013	1	1	1807.0	29.0	2251.0	NaN	UA	N31412	1228	EWB	SAN	NaN	2425	18.0	7.0
403	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EHAA	791	LGA	DFW	NaN	1389	NaN	NaN
404	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EVAA	1925	LGA	MIA	NaN	1096	NaN	NaN
855	2013	1	2	2145.0	16.0	NaN	NaN	UA	N12221	1299	EWB	RSW	NaN	1068	21.0	45.0
858	2013	1	2	NaN	NaN	NaN	NaN	AA	NaN	133	JFK	LAX	NaN	2475	NaN	NaN

Notebook

IMPUTATION

- A method to “fill in” missing value
- Use other variables (which have values) to predict value on missing variable
- Involves building a model for variable with missing value
 - $Y = f(x_1, x_2, \dots, x_n)$

Methods will be covered later

MISSING VALUES

- When summing the data, missing values will be treated as zero
- If all values are missing, the sum will be equal to NaN
- `cumsum()` and `cumprod()` methods ignore missing values but preserve them in the resulting arrays
- Missing values in `GroupBy` method are excluded (just like in R)
- Many descriptive statistics methods have *skipna* option to control if missing data should be excluded . This value is set to *True* by default (unlike R)

BASIC DESCRIPTIVE STATISTICS

df.method()	description
describe	Basic statistics (count, mean, std, min, quantiles, max)
min, max	Minimum and maximum values
mean, median, mode	Arithmetic average, median and mode
var, std	Variance and standard deviation
sem	Standard error of mean
skew	Sample skewness
kurt	kurtosis

METADATA

- Data about data
- Detailed description of the variables in the file, their meaning and permissible values

Marital Status Value	Description
1	Married, data from source 1
2	Single, data from source 1
4	Divorced, data from source 1
D	Divorced, data from source 2
M	Married, data from source 2
S	Single, data from source 2
Blank	Marital status is missing

READING MATERIAL

- Python documentation: <https://docs.python.org/3/tutorial/index.html>
- NumPy: https://numpy.org/doc/stable/user/absolute_beginners.html
- Pandas: https://pandas.pydata.org/docs/getting_started/intro_tutorials/index.html
- Wrangling Dataframes: https://learningds.org/ch/09/wrangling_intro.html